

Module 4: Replication and Coherence

4.1 Introduction to Replication

Replication involves maintaining copies of data across multiple servers. This massively increases data availability, fault tolerance, and system performance by bringing data closer to the user (a technique also known as caching-at-scale). However, the core challenge is **consistency** — ensuring that an update to one replica is accurately reflected across all other replicas, ideally before anyone notices a difference.

This is closely related to the well-known **CAP theorem**, which states that in the presence of a network partition, a system must choose between **Consistency** (every read gets the latest write) and **Availability** (every request gets a response, even a possibly stale one). You cannot have perfect versions of both at the same time during a partition.

4.2 Data-Centric and Client-Centric Consistency Models

Data-Centric Consistency focuses on the rules governing the data store itself, independent of any one user:

Model	Guarantee
Strict Consistency	Every read returns the absolute latest write, instantly — the strongest (and most expensive) guarantee.
Sequential Consistency	All processes see operations in the same order, though not necessarily the real-time order.
Causal Consistency	Operations that are causally related are seen in the same order by everyone; unrelated operations may be seen differently.

Client-Centric Consistency instead focuses on the guarantees given to one specific, moving user as they interact with different replicas over time:

Guarantee	What It Promises the User
Read-your-writes	A user always sees their own most recent update, even after switching replicas.
Monotonic reads	Once a user has seen a value, they will never see an older value later.
Monotonic writes	A user's writes are applied in the order the user issued them.
Writes-follow-reads	A write made after reading some value is applied after that value everywhere.

■ Read-Your-Writes in Practice

You update your profile photo on a social media app while on Wi-Fi. You then switch to mobile data, which routes you to a different, geographically closer server replica. Read-your-writes consistency guarantees you will still see your new photo, not the old one, even though you are now talking to a completely different machine.

4.3 Consistency Protocols

To enforce these models, systems deploy concrete protocols:

- **Primary-Based Protocols:** A master server handles all writes, which are then cascaded (pushed) to read-only replicas. Simple to reason about, but the primary can become a bottleneck.
- **Replicated-Write (Quorum-Based):** Updates can happen anywhere, but a transaction must receive approval from a "quorum" (a strict majority) of nodes before being committed, ensuring overlapping consistency.

Quorum systems are often summarized with a simple formula. If there are N replicas, a read quorum of R nodes and a write quorum of W nodes, consistency is guaranteed whenever $R + W > N$ — because any read set and any write set are then guaranteed to overlap on at least one up-to-date node.

■ Key Takeaway

Replication buys you speed and resilience, but every replication scheme is really just a different answer to the question: "how stale is an acceptable answer, and for how long?"

Glossary of Key Terms

Term	Meaning
Replica	A copy of data stored on a different server.
CAP theorem	Under a network partition, a system trades off Consistency vs Availability.
Quorum	The minimum number of replicas that must agree before an operation is considered valid.
Primary-based protocol	A scheme where one designated node handles all writes.

What You Should Know

- The inherent trade-off between replication for performance/availability and the overhead of maintaining consistency (and how CAP frames this trade-off).
- The differences between Data-centric models (strict, sequential, causal) and Client-centric models (read-your-writes, monotonic reads/writes, writes-follow-reads).
- How Quorum-based voting ($R + W > N$) guarantees reliable writes in a decentralized setup.